Class Notes – GitHub Token & API Usage

Prepared by: AediaX Tech Private Limited Date: 04 April 2026

1. What is a GitHub Token?

A GitHub Token, also known as a Personal Access Token (PAT), is a secure authentication key used to access GitHub services and APIs without using a password.

It is mainly used in: - Applications - Backend servers - Automation scripts - CI/CD pipelines

Key Characteristics

- Acts like a password but is safer
- Can be restricted with permissions (scopes)
- Can be expired or revoked anytime

Example Token

ghp_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

2. Why Use GitHub Token Instead of Password?

- GitHub no longer allows password authentication for APIs
- Tokens are more secure
- You can control access using scopes
- Easy to revoke if compromised

3. How to Generate a GitHub Token

Steps

1. Login to GitHub
2. Go to Profile → Settings
3. Scroll to Developer Settings
4. Click on Personal Access Tokens
5. Choose Tokens (classic)
6. Click on “Generate new token”

Configure Token

- Name: Example → MyAppToken
- Expiration: Set a validity period
- Select Scopes (permissions)

Important Scopes

- repo → Full control of repositories
- read:repo → Read-only access
- workflow → GitHub Actions

Final Step

- Click Generate Token
- Copy and store securely

Note: Token will be shown only once

4. Using GitHub Token in API

GitHub provides REST APIs to interact with repositories.

Authentication Format

Authorization: token YOUR_GITHUB_TOKEN

5. Create/Update File Using GitHub API

API Endpoint

PUT https://api.github.com/repos/{owner}/{repo}/contents/{path}

Example JavaScript Code

```
fetch("https://api.github.com/repos/USERNAME/REPO/contents/data.json", {
  method: "PUT",
  headers: {
    "Authorization": "token YOUR_GITHUB_TOKEN",
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    message: "Add or update data",
    content: btoa(JSON.stringify({ name: "Amresh", age: 22 }))
  })
})
```

```
.then(res => res.json())
.then(data => console.log(data));
```

Explanation

- message → Commit message
- content → Base64 encoded data
- path → File location in repository

6. Reading Data from GitHub

API Endpoint

GET https://api.github.com/repos/{owner}/{repo}/contents/{path}

Example

```
fetch("https://api.github.com/repos/USERNAME/REPO/contents/data.json")
  .then(res => res.json())
  .then(data => {
    const content = JSON.parse(atob(data.content));
    console.log(content);
  });
```

7. Using GitHub as a Database

GitHub can be used as a simple storage system by saving structured data (JSON files) inside a repository.

Working Flow

1. Store data in JSON file
2. Use API to read/write
3. Each change creates a commit

Example Use Cases

- Small projects
- Static CMS
- Config storage
- Demo applications

8. Advantages of Using GitHub as DB

- Free storage

- Version control (history tracking)
- Easy integration with APIs
- No server required for small apps

9. Limitations of Using GitHub as DB

- Not real-time
- API rate limits
- Slower compared to databases
- Not suitable for large-scale apps
- No advanced querying like SQL/NoSQL

10. Security Best Practices

- Never expose token in frontend code
- Always use backend to call APIs
- Store token in environment variables (.env)
- Use minimum required permissions
- Rotate tokens regularly

Final Summary

- GitHub Token is used for secure authentication
- Required for API-based operations
- Helps automate file creation and updates
- GitHub can act as a basic database using JSON files
- Not suitable for production-scale applications